CS 412 / 512
Introduction to Computer Graphics

Zhu Wang
Assistant Professor of Computer Science



COMPUTER SCIENCE

Special Topic: Procedural Generation and Perlin Noise

Dr. Ken Perlin

CS Colloquium Talk on Wednesday September 30, 2026 (CEC 1041)

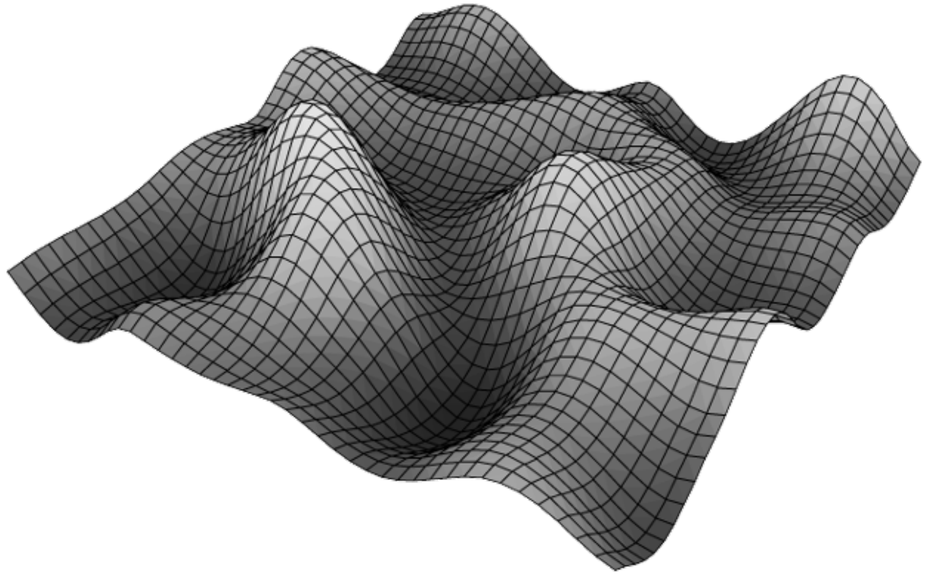






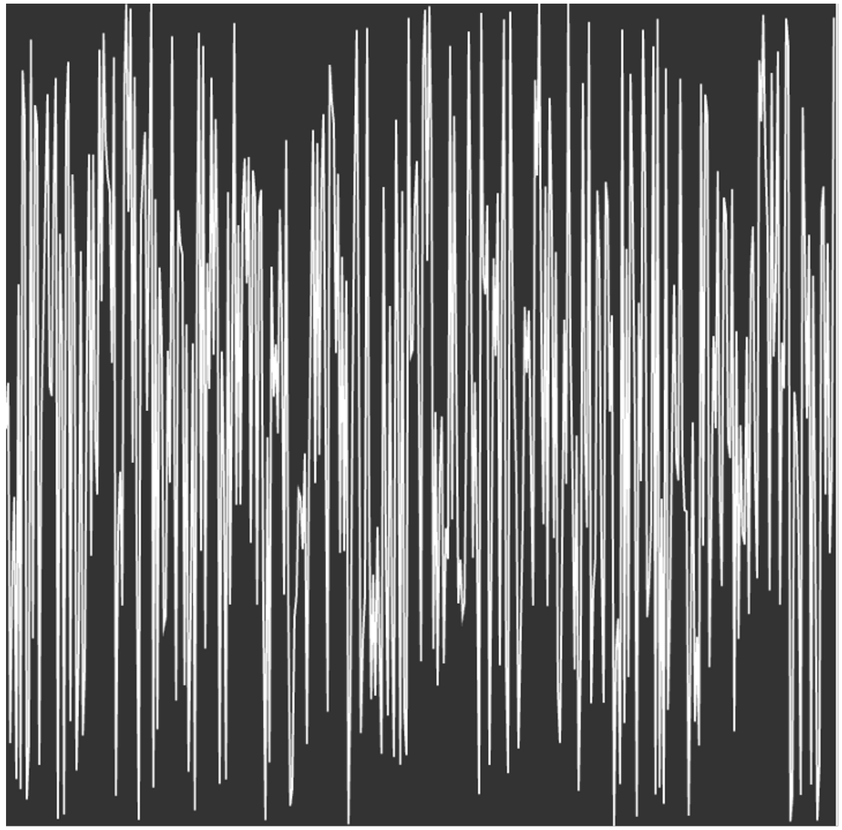
Procedural Generation

It is a method of creating digital content algorithmically instead of manually, often used in generating maps, terrains, game levels, characters, and other elements that require unique and unpredictable results

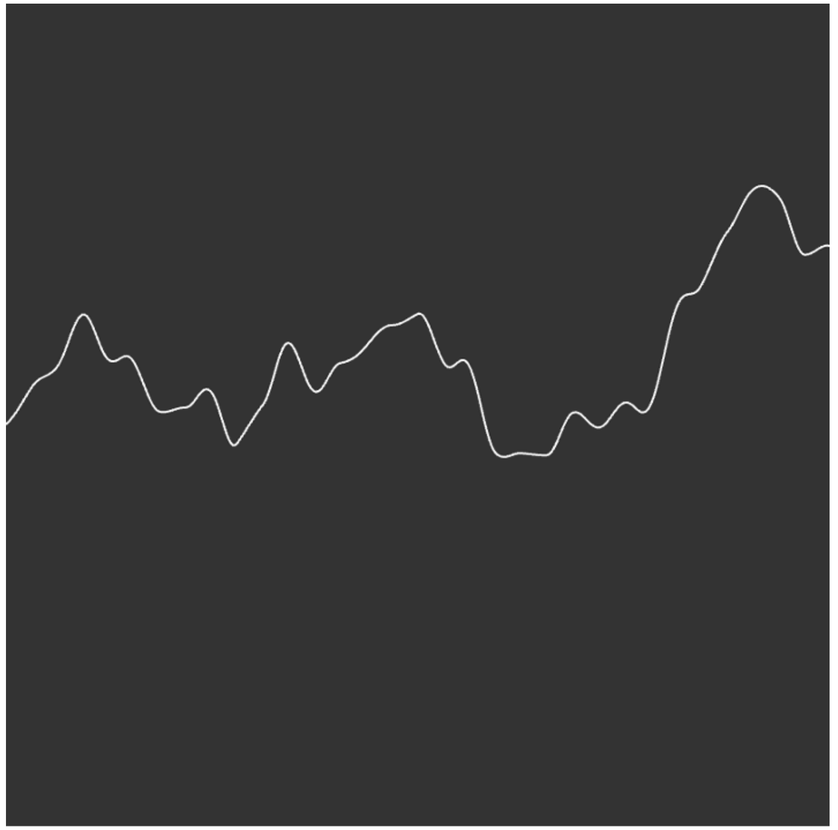




Regular Random Noise vs Perlin Noise



Regular Random Noise

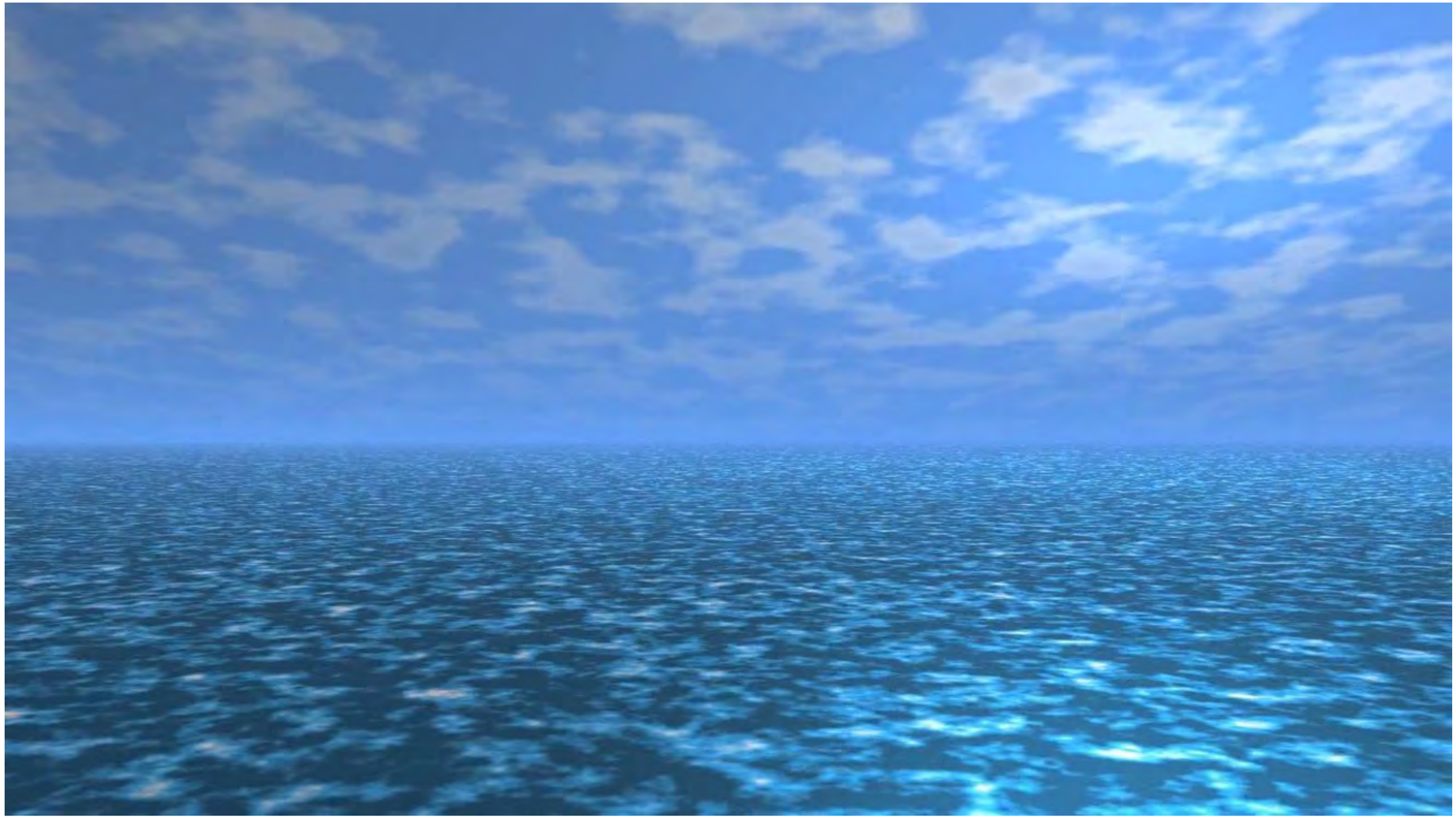


Perlin Noise

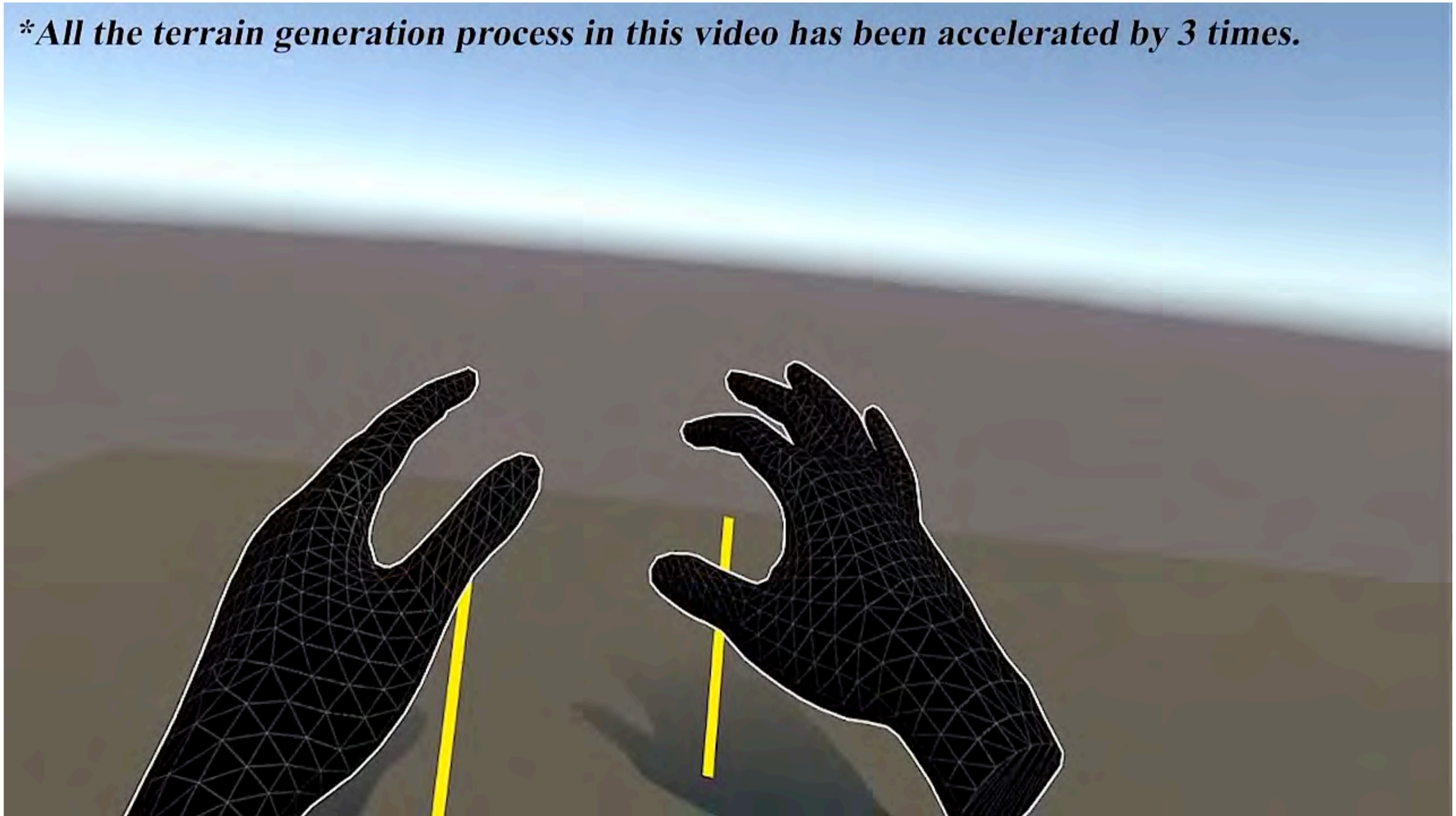
Terrain/Level Generation for Minecraft



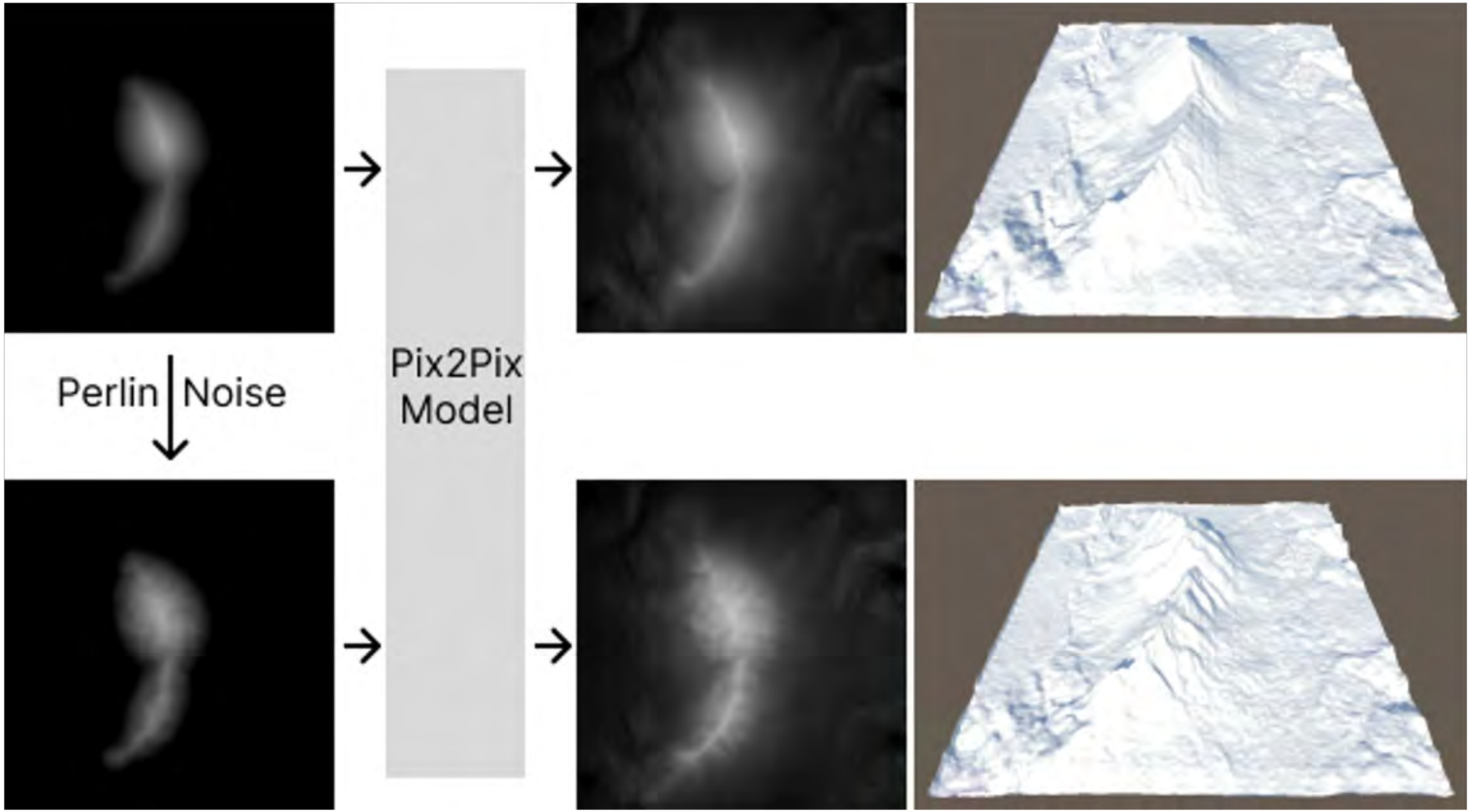
Cloud and Water



Terrain Generation



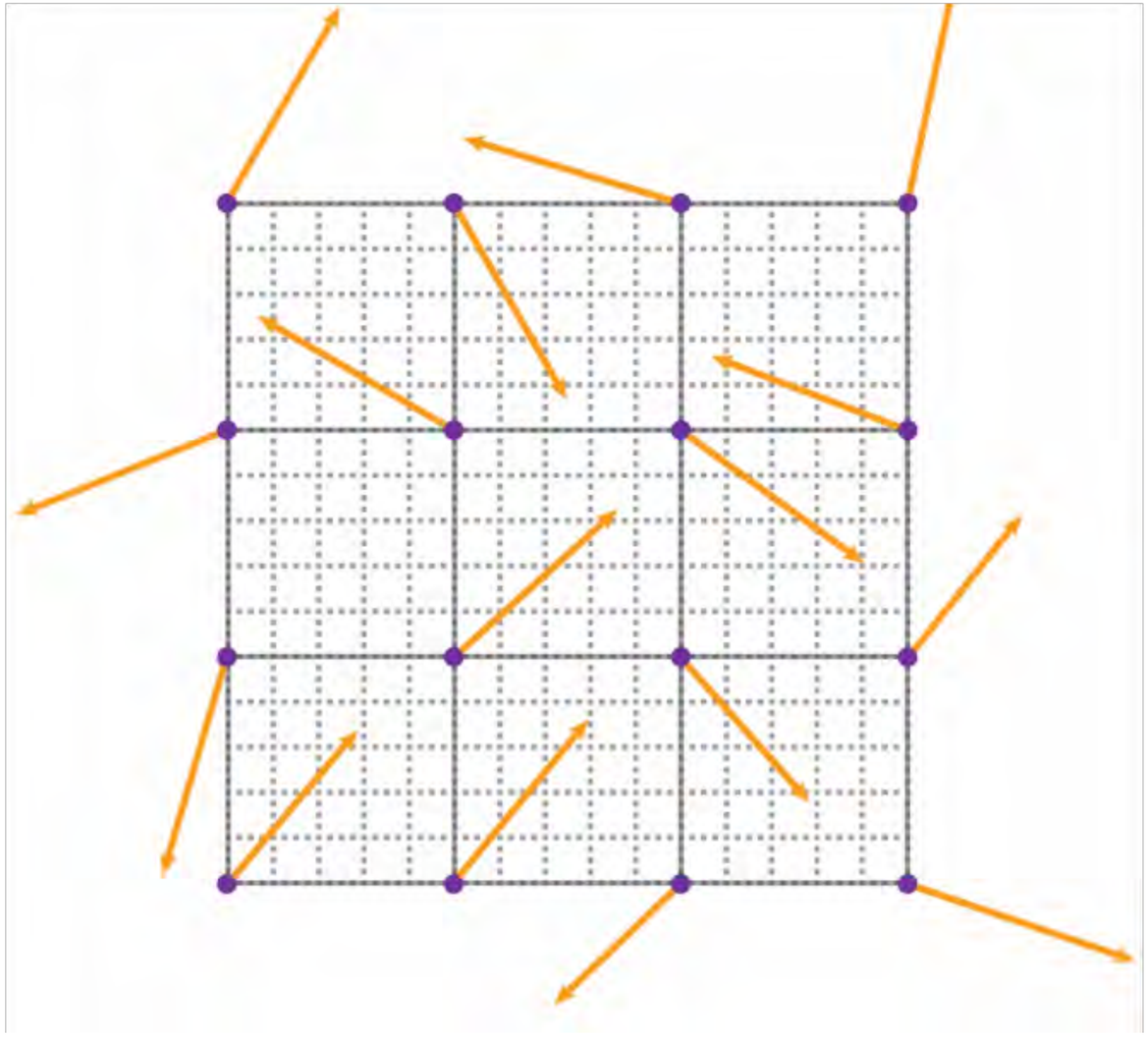
Terrain Generation



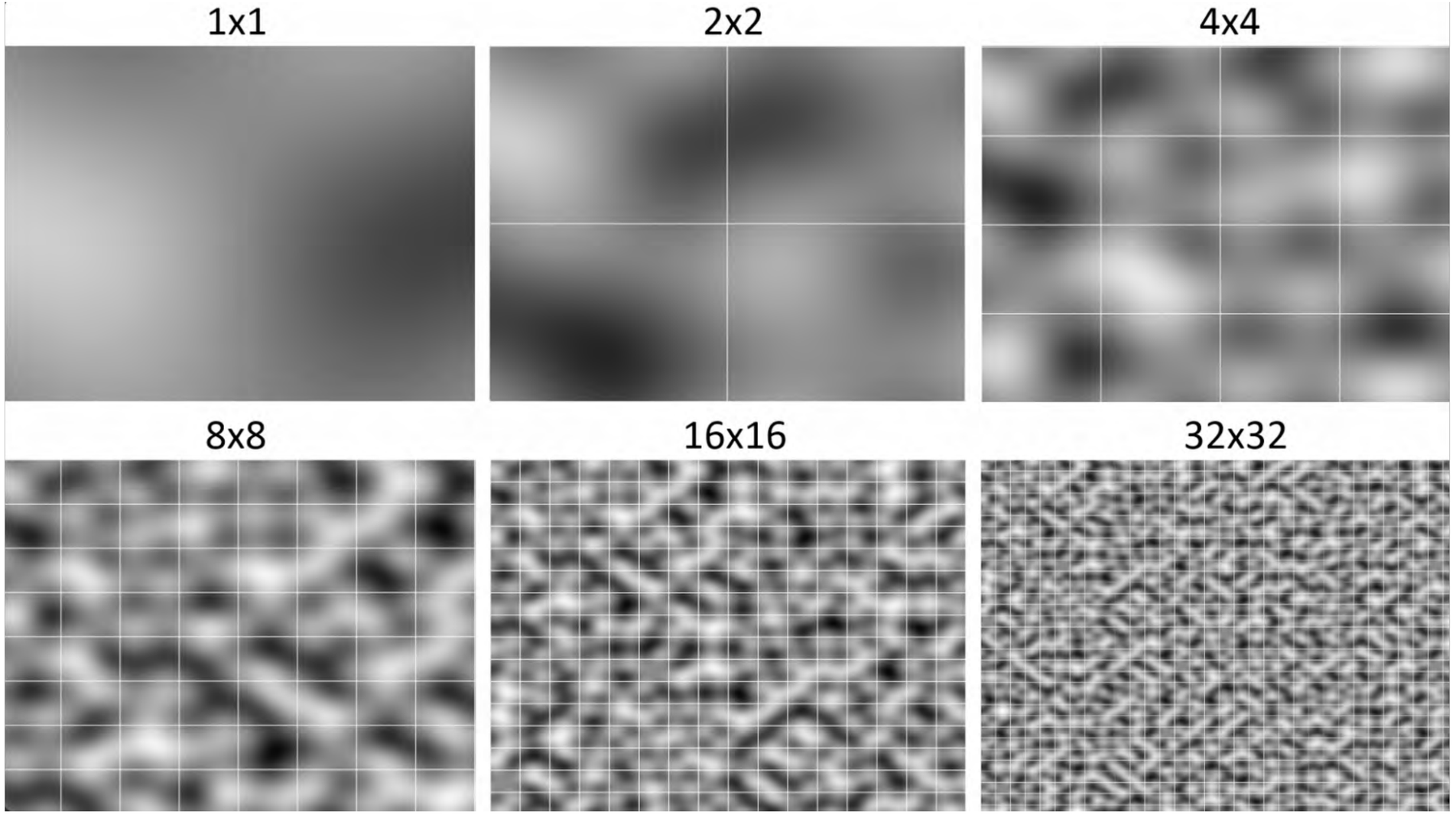
Perlin Noise

1. Define the lattice

2. Assign random vector direction to each corner

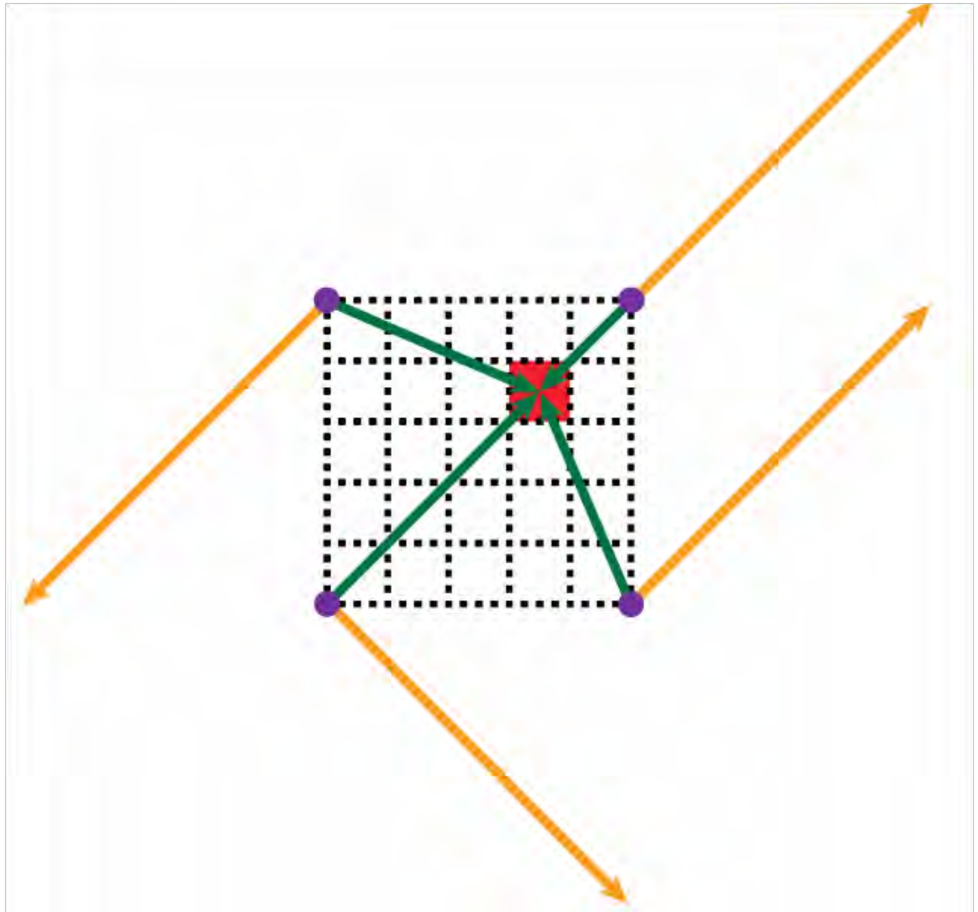


Different Grid Sizes



Dot Product and Interpolation

3. Dot product and blending of the gradient vector the offset vectors



4. Interpolate the value with the fade function for each axis:

$$f(t) = 6t^5 - 15t^4 + 10t^3$$

$$f(0) = 0,\ f(1) = 1$$

$$f'(0) = f'(1) = 0 \qquad f''(0) = f''(1) = 0$$

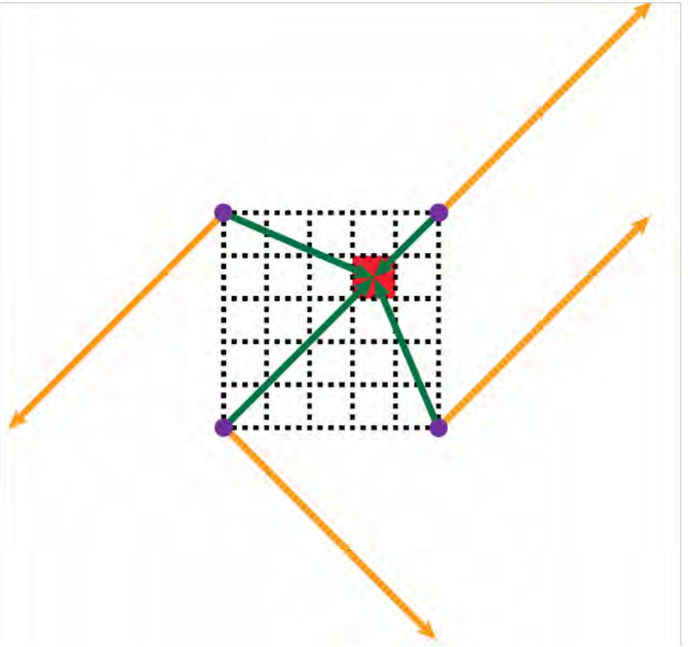
Dot Product and Interpolation

For 2D images

$$x_i = \lfloor x \rfloor, \quad t_x = x - x_i \qquad u = f(t_x)$$

$$y_i = \lfloor y \rfloor, \quad t_y = y - y_i \qquad v = f(t_y)$$

$$n_{00} = g(x_i, y_i) \cdot (t_x, t_y)$$
$$n_{10} = g(x_i + 1, y_i) \cdot (t_x - 1, t_y)$$
$$n_{01} = g(x_i, y_i + 1) \cdot (t_x, t_y - 1)$$
$$n_{11} = g(x_i + 1, y_i + 1) \cdot (t_x - 1, t_y - 1)$$



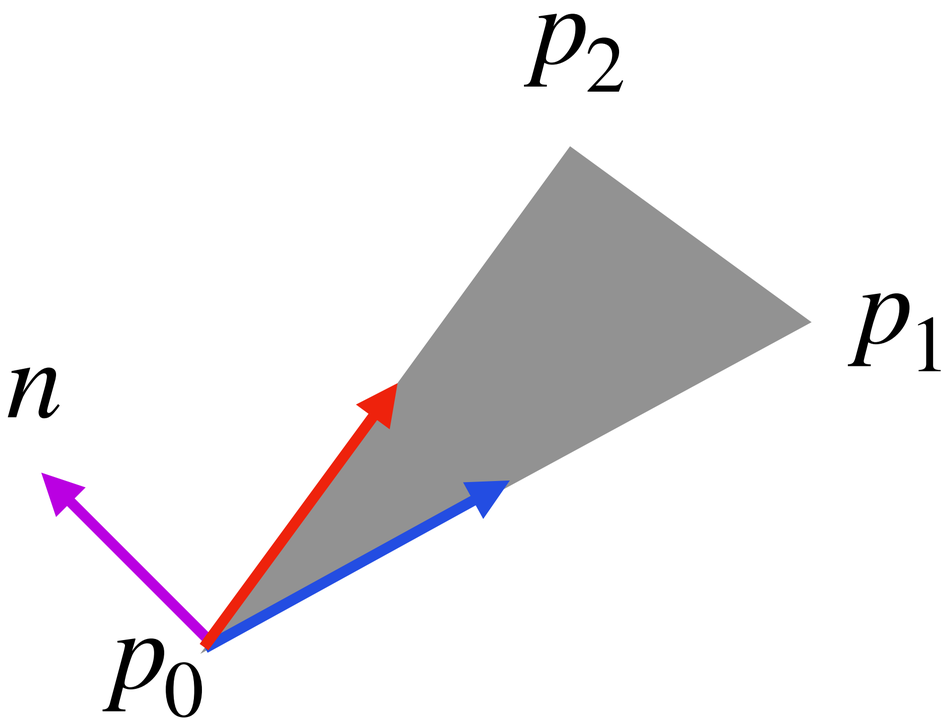
g(i, j) assigns a fixed pseudo-random gradient vector to each integer grid point (usually from (±1, ±1), (±1, 0), (0, ±1)), determining the direction in which the noise increases near that corner.

$$P(x, y) = (1 - u)(1 - v)\, n_{00} + u(1 - v)\, n_{10} + (1 - u)\, v\, n_{01} + u\, v\, n_{11}$$

Shadings and Normals

Computing Normals

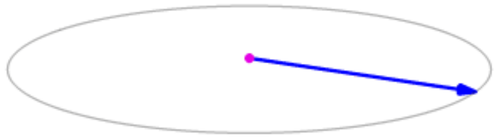
Normal vectors on a triangle

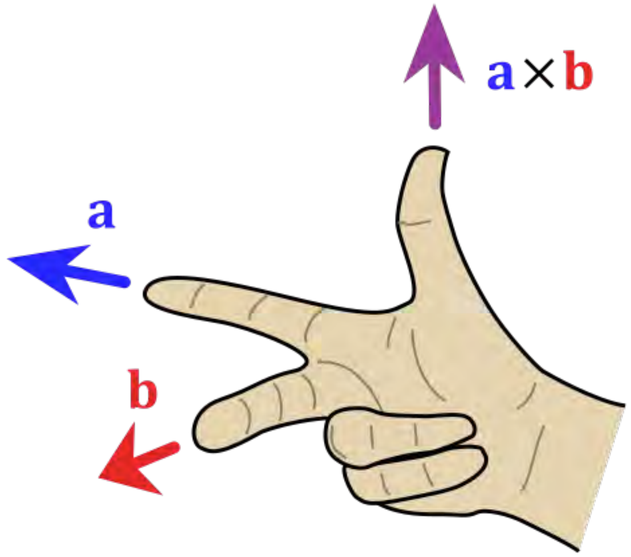


Cross product between two edge vectors

$$n = (p_1 - p_0) \times (p_2 - p_0)$$

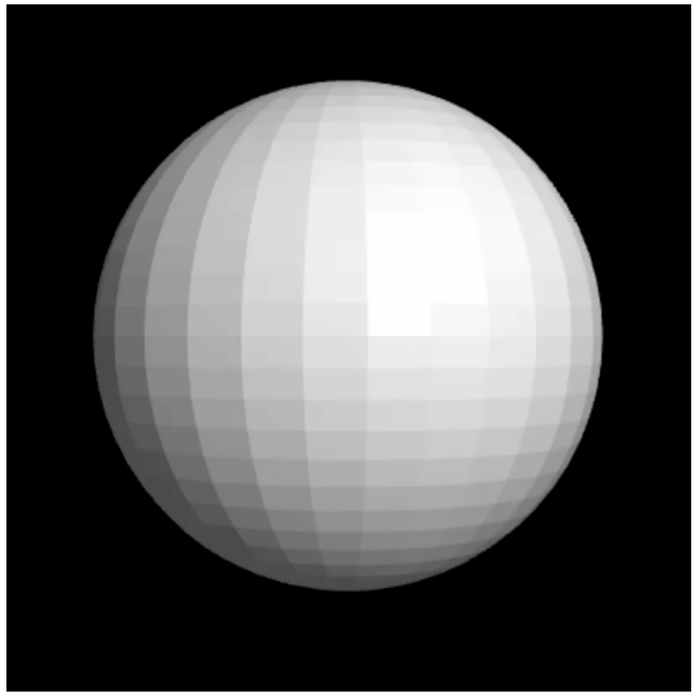
Normalize $\quad n = \dfrac{n}{\|n\|}$



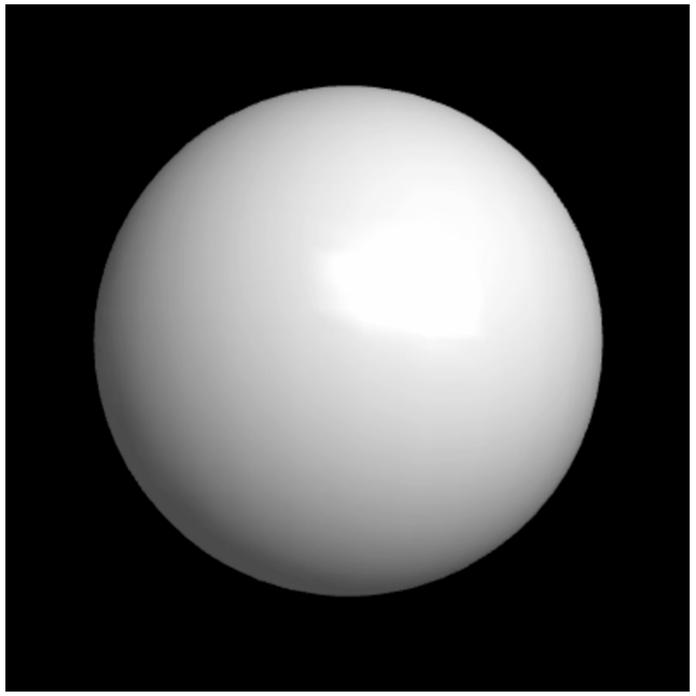


Right-hand rule

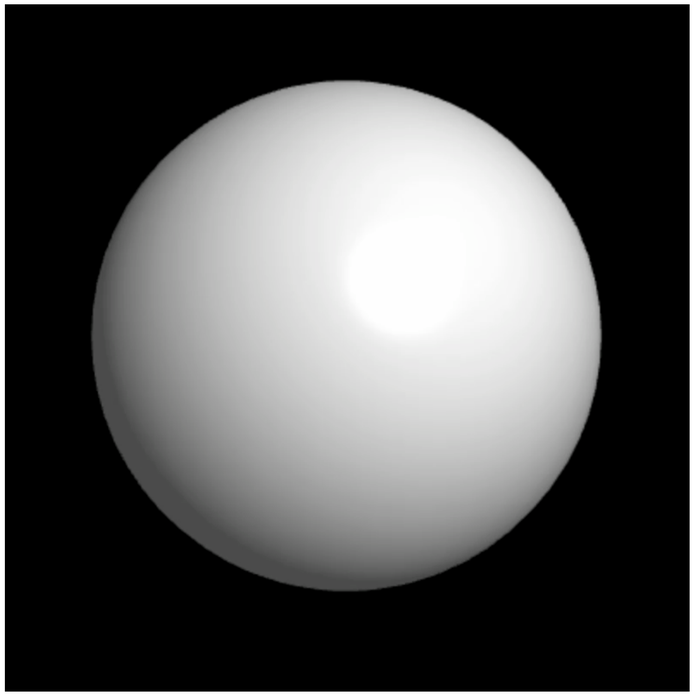
Flat and Smooth Shadings



Flat shading
(Face normals)

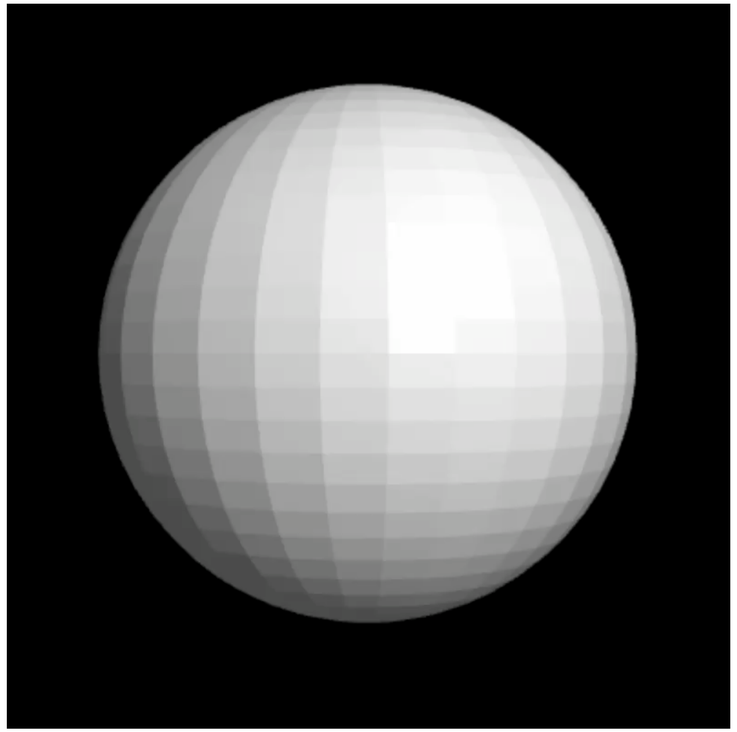


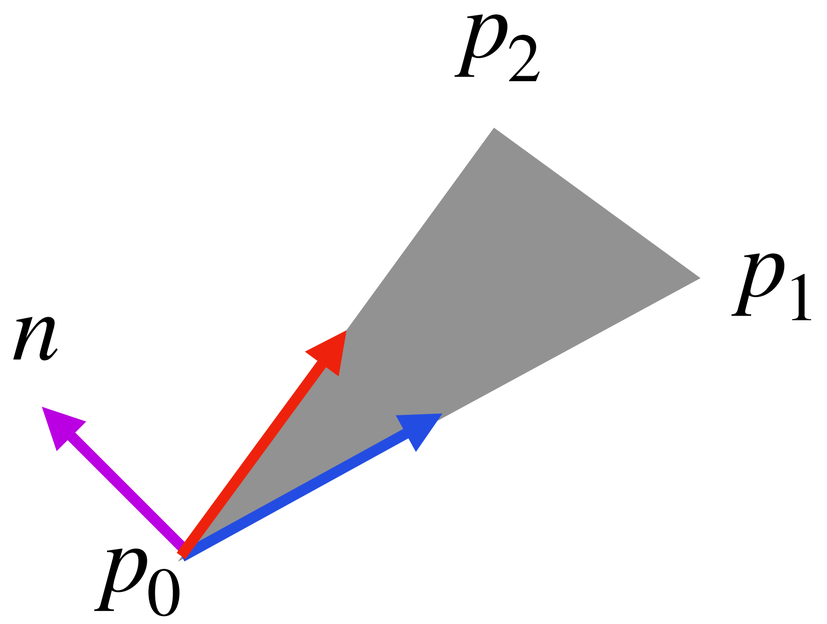
Gouraud shading
(Per-vertex)



Phong shading
(per-pixel/per-fragment)

Flat Shading





For each of the triangles on the sphere:

$$n = (p_1 - p_0) \times (p_2 - p_0)$$

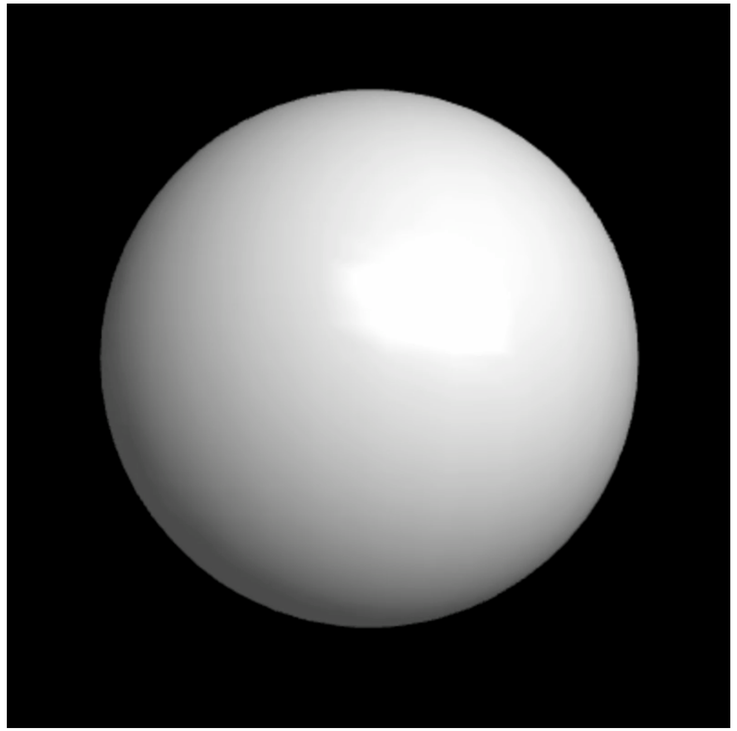
$$n = normalize(n)$$

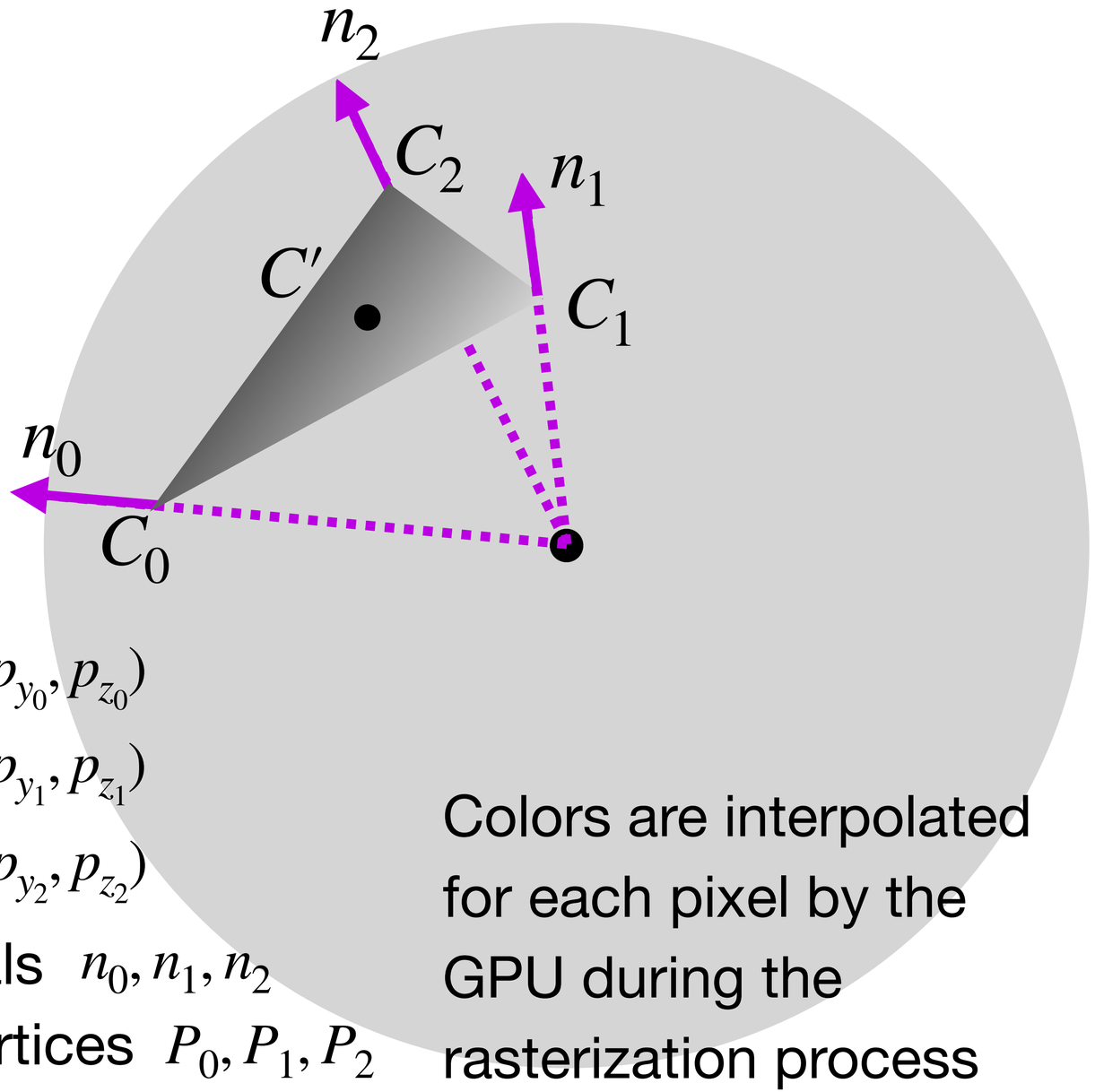
Set n as the normal vector for all three vertices P_0, P_1, P_2

Same normal for the entire triangle

Gouraud Shading

Interpolating colors, not normals





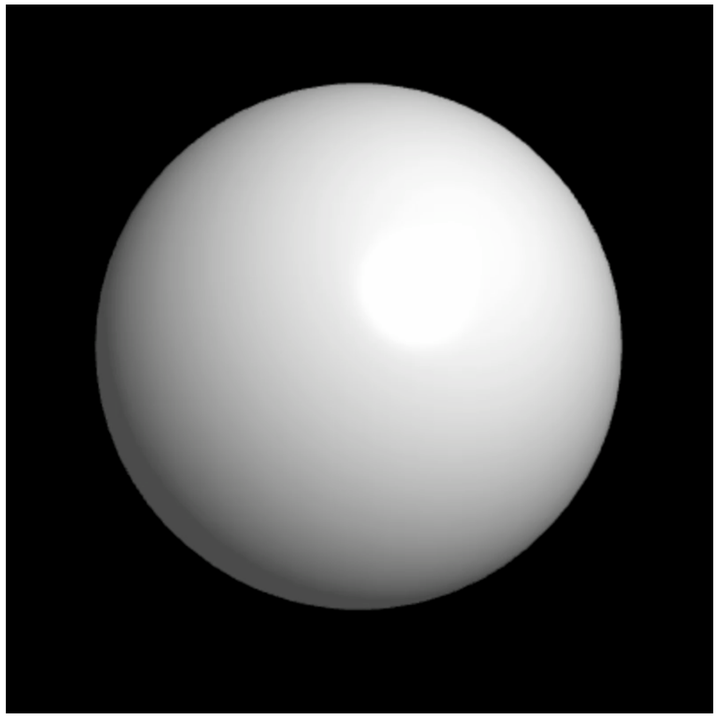
$$n_0 = (p_{x_0}, p_{y_0}, p_{z_0})$$
$$n_1 = (p_{x_1}, p_{y_1}, p_{z_1})$$
$$n_2 = (p_{x_2}, p_{y_2}, p_{z_2})$$

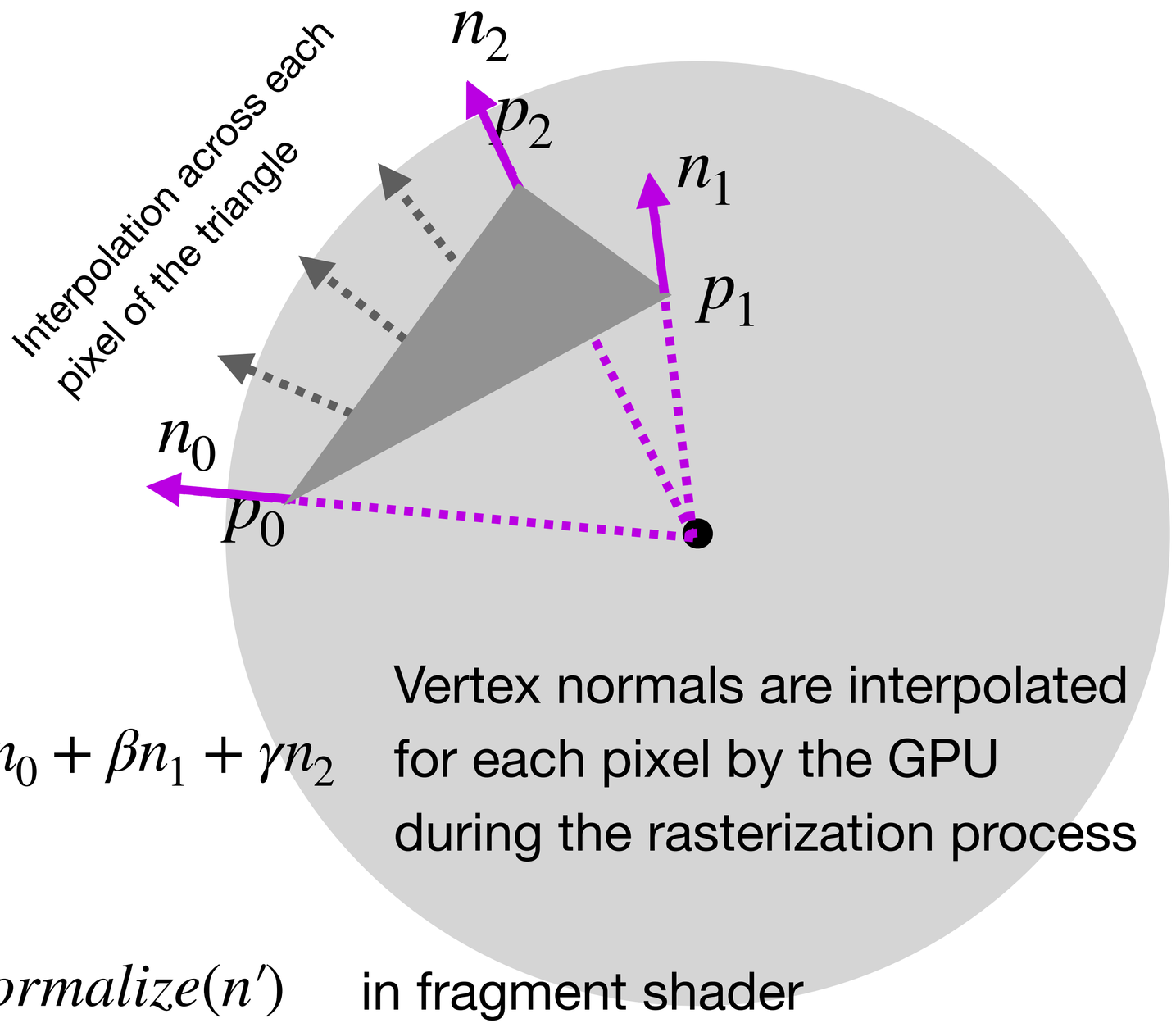
Different normals n_0, n_1, n_2
for the three vertices P_0, P_1, P_2

Colors are interpolated for each pixel by the GPU during the rasterization process

Phong Shading

Interpolating normals





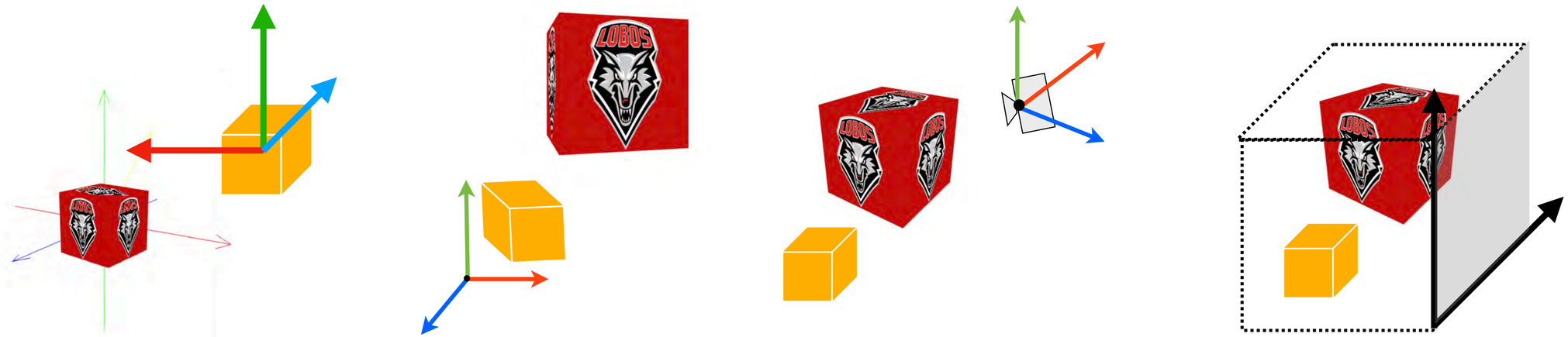
$$n' = \alpha n_0 + \beta n_1 + \gamma n_2$$

Vertex normals are interpolated for each pixel by the GPU during the rasterization process

$$n' = normalize(n')$$

in fragment shader

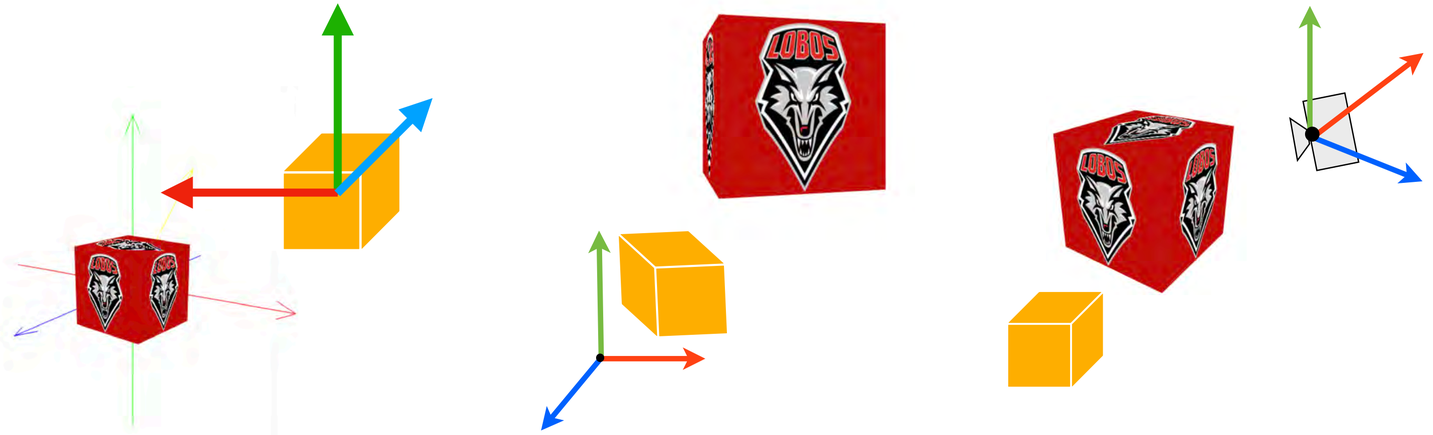
Shading Transformations



Most common to do it here

You can do it in any stage, but each has trade-offs.

Shading Transformations



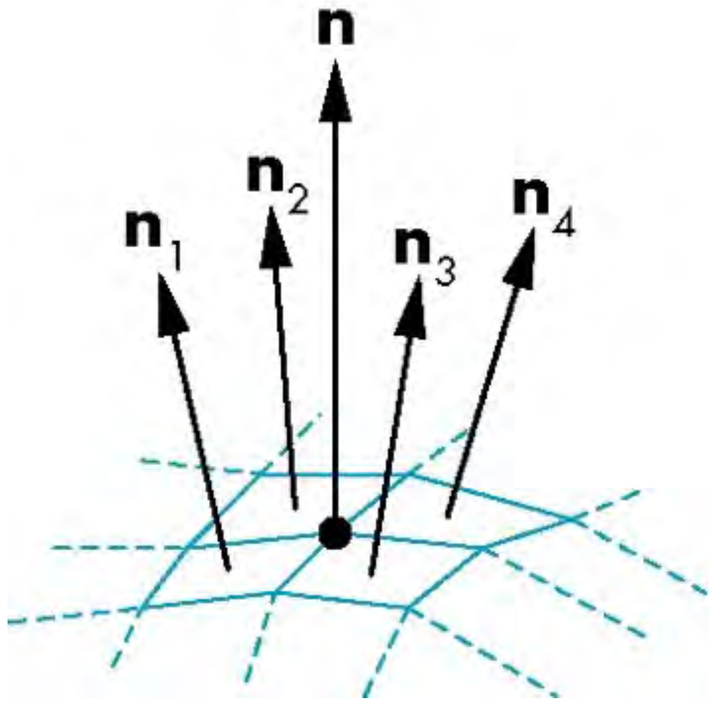
Model coordinates → Camera coordinates

$p' = Mp$ where M is model-view transformation

$n' = M_s n$ where M_s is shading transformation, it is usually 3x3 matrix

Averaging Normals for Smooth Shading

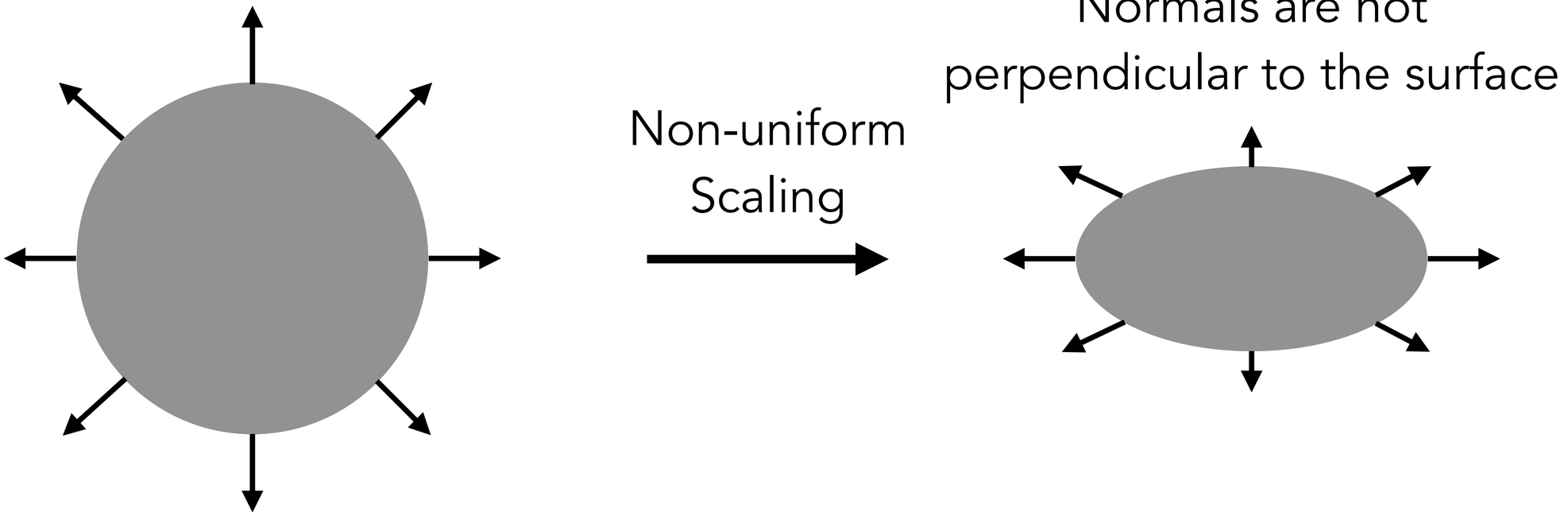
Smooth shading from an existing 3D model with flat shading



[source: Textbook by Prof. Ed Angel]

$$\mathbf{n} = \frac{\sum_i \mathbf{n}_i}{\left\| \sum_i \mathbf{n}_i \right\|}$$

When Scaling Breaks Your Normals



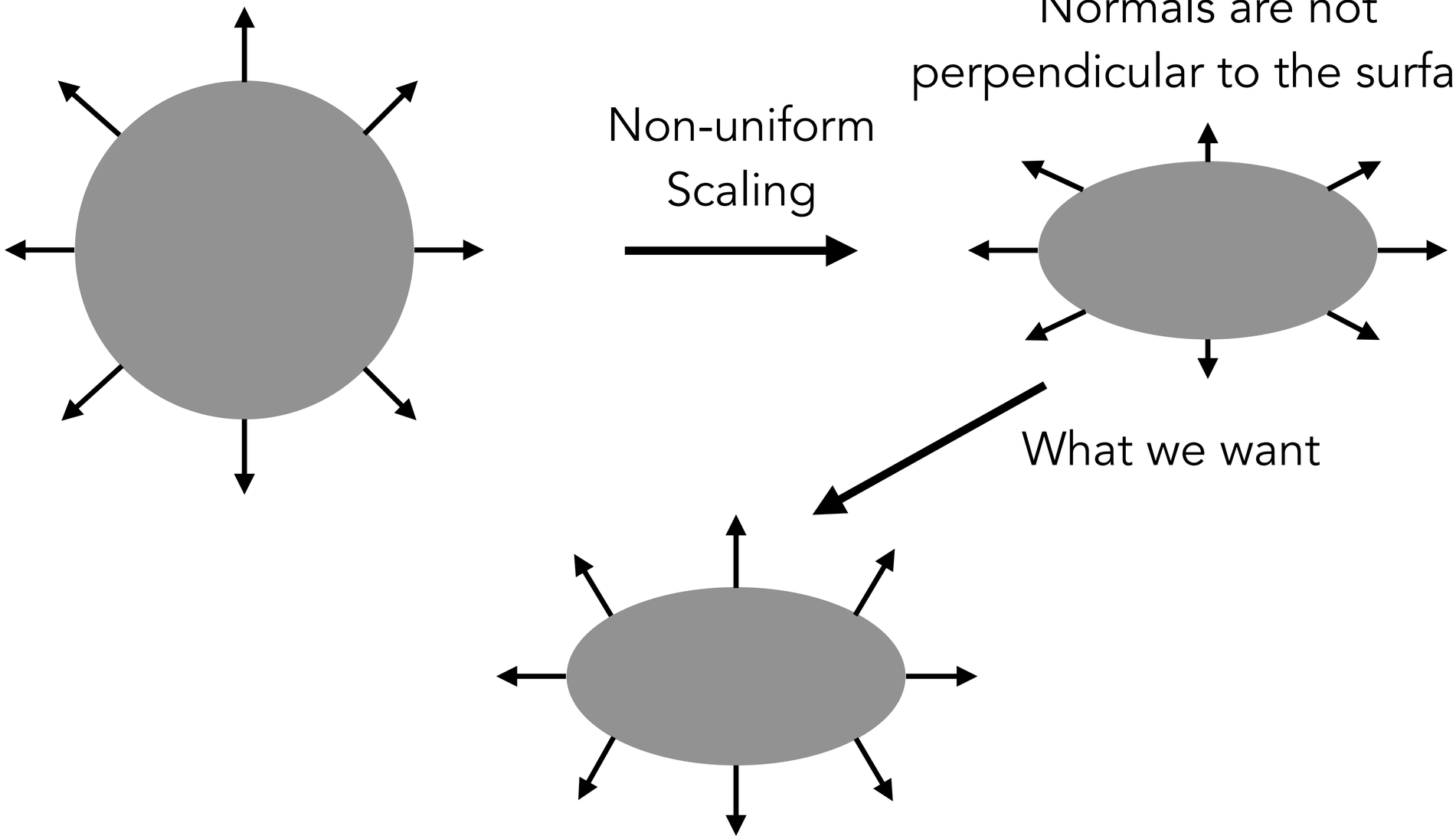
Non-Uniform Scaling Distorts Normals

Non-uniform scaling could cause the transformed normals no longer perpendicular to the surface.

Positions move with the surface. But normals must stay perpendicular to that surface after transformation.

So we need to apply some correction to normals.

Transforming Normals



Inverse Transpose Transformation

$$a \cdot (Mb) = a^T(Mb)$$

$n \cdot t = 0$ where t is a tangent vector lying on the surface

We want n' to satisfy

$t' = Mt$ ⟶ $n' \cdot t' = 0$

$$n' \cdot (Mt) = 0$$

$$(M^T n') \cdot t = 0$$

$$(M^T n') = n$$

$$n' = (M^{-1})^T n \qquad M_s = (M^{-1})^T$$

fundamental property of the dot product

$$a \cdot (Mb) = a^T(Mb) \qquad a \cdot (Mb) = a^T(Mb) = (a^T M)b = (M^T a)^T b = (M^T a) \cdot b$$

What About Arbitrary Transformations?

Singular Value Decomposition (SVD) $M = U\Sigma V^T$

$n \cdot t = 0$ where t is a tangent vector lying on the surface

When M, M_s are 3x3

$$t' = Mt = U\Sigma V^T t$$

We want n' and M_s to satisfy

$$n' \cdot t' = 0 \qquad M_s = R_1 S^{-1} R_2$$

$$M^{-1} = (R_1 S R_2)^{-1} = R_2^{-1} S^{-1} R_1^{-1}$$

$$\begin{aligned}(M^{-1})^T &= (R_2^{-1} S^{-1} R_1^{-1})^T \\ &= (R_2^T S^{-1} R_1^T)^T \\ &= R_1 (R_2^T S^{-1})^T \\ &= R_1 (S^{-1})^T R_2 \\ &= R_1 S^{-1} R_2\end{aligned}$$

Singular Value Decomposition (SVD)

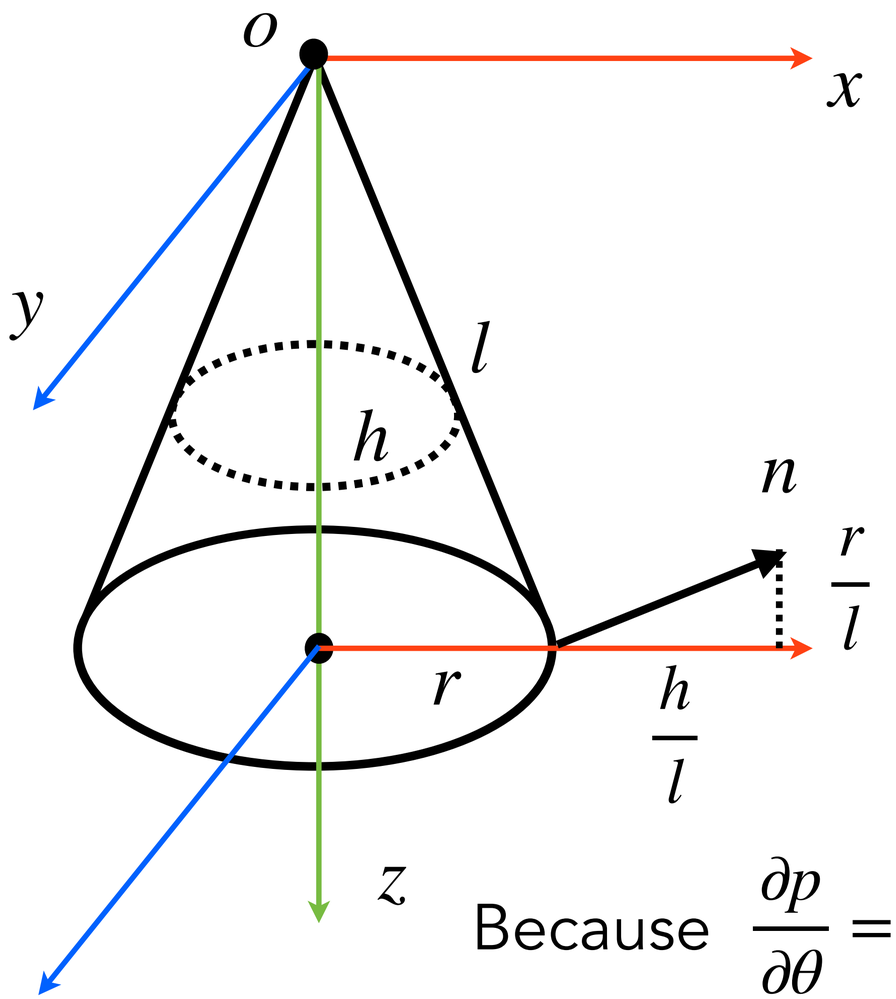
$$M = U\Sigma V^T$$

M can be decomposed as

$$M = R_1 S R_2$$

Geometries with Curvatures

Cone



Apex at the origin, base is on the plane $z = h$

$\theta \in [0,2\pi)$ the angle around z-axis

$u \in [0,1]$ the position moving along z-axis

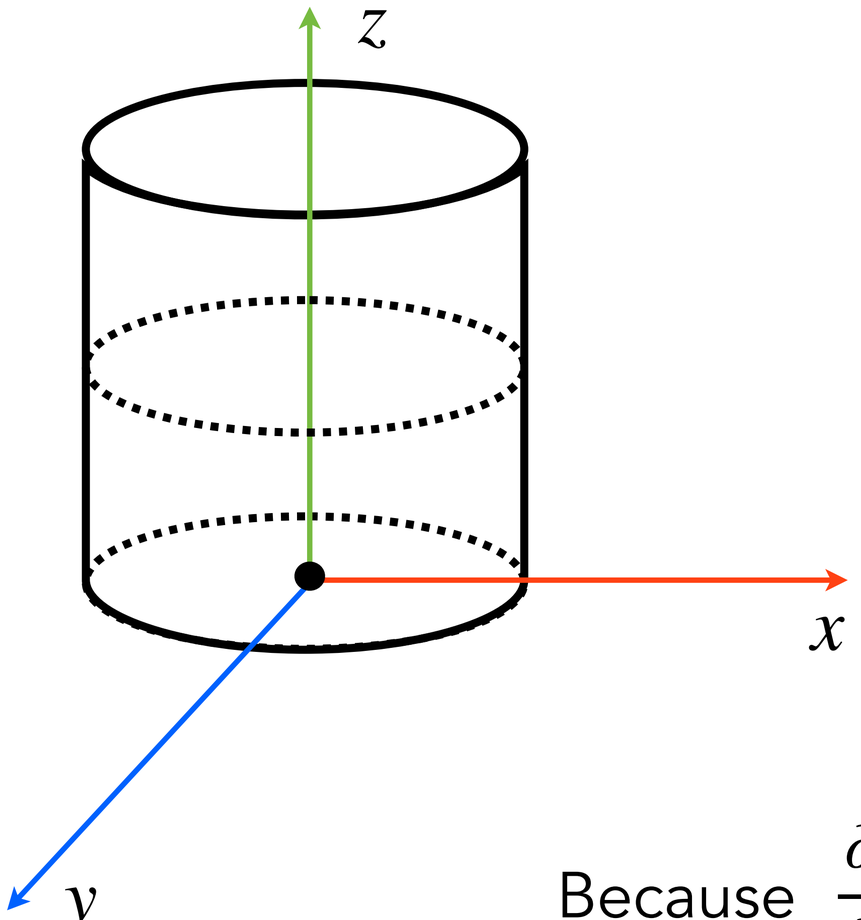
$$p(\theta, u) = \begin{bmatrix} rucos\theta \\ rusin\theta \\ h(1-u) \end{bmatrix}$$

normalized

$$\hat{n} = \frac{1}{\sqrt{r^2 + h^2}} \begin{bmatrix} hcos\theta \\ hsin\theta \\ r \end{bmatrix}$$

Because $\frac{\partial p}{\partial \theta} = \begin{bmatrix} -rusin\theta \\ rucos\theta \\ 0 \end{bmatrix}$ $\frac{\partial p}{\partial u} = \begin{bmatrix} rcos\theta \\ rsin\theta \\ -h \end{bmatrix}$ $n = \frac{\partial p}{\partial \theta} \times \frac{\partial p}{\partial u} = ru \begin{bmatrix} hcos\theta \\ hsin\theta \\ r \end{bmatrix}$

Cylinder



Apex at the origin, base is on the plane $z = h$

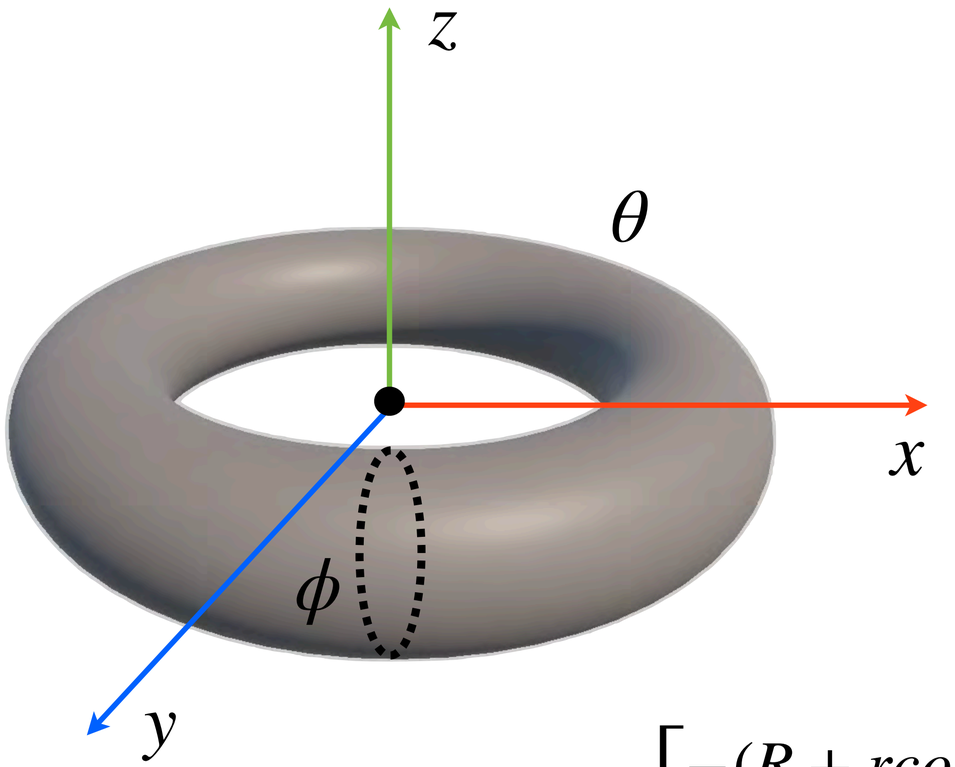
$\theta \in [0, 2\pi)$ the angle around z-axis

$z \in [0, h]$ the position moving along z-axis

$$p(\theta, z) = \begin{bmatrix} r\cos\theta \\ r\sin\theta \\ z \end{bmatrix} \quad \text{normalized} \quad \hat{n} = \begin{bmatrix} \cos\theta \\ \sin\theta \\ 0 \end{bmatrix}$$

Because $\frac{\partial p}{\partial \theta} = \begin{bmatrix} -r\sin\theta \\ r\cos\theta \\ 0 \end{bmatrix}$ $\frac{\partial p}{\partial z} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$ $n = \frac{\partial p}{\partial \theta} \times \frac{\partial p}{\partial u} = \begin{bmatrix} r\cos\theta \\ r\sin\theta \\ 0 \end{bmatrix}$

Torus/Donut



R large circle radius r tube radius

$\theta \in [0,2\pi)$ the angle around large circle

$\phi \in [0,2\pi)$ The angle around the tube

$$p(\theta,\phi) = \begin{bmatrix} (R + rcos\phi)cos\theta \\ (R + rcos\phi)sin\theta \\ rsin\phi \end{bmatrix}$$

normalized

$$\hat{n} = \begin{bmatrix} cos\phi cos\theta \\ cos\phi sin\theta \\ sin\phi \end{bmatrix}$$

Because $\dfrac{\partial p}{\partial \theta} = \begin{bmatrix} -(R + rcos\phi)sin\theta \\ (R + rcos\phi)cos\theta \\ 0 \end{bmatrix}$ $\dfrac{\partial p}{\partial z} = \begin{bmatrix} -rsin\phi cos\theta \\ -rsin\phi sin\theta \\ rcos\phi \end{bmatrix}$ $n = \dfrac{\partial p}{\partial \theta} \times \dfrac{\partial p}{\partial u}$